MateOrg（MaterialOrganization）
ファイル構成・保守ガイド

対象ブランチ: `migrate-to-nextjs`（コミット 8ae53e2 時点）／ 作成日: 2026-10-20

対象: `frontend/`（Next.js）と `backend/`（Rails API）。`node_modules/`・`.next/` などの自動生成物は除く。

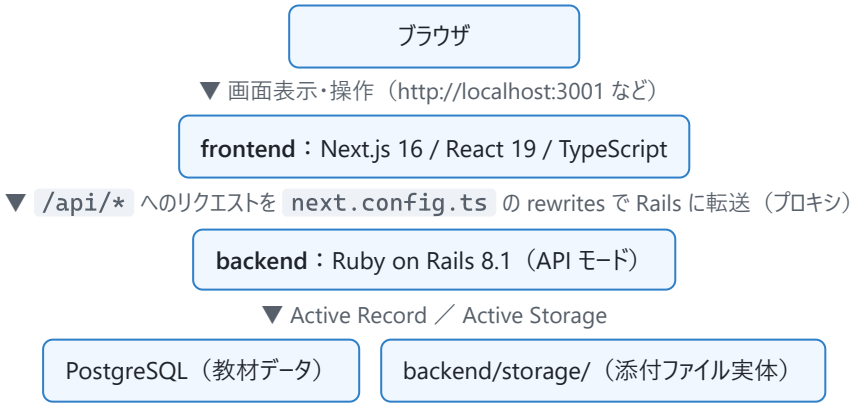
目次

1. システム全体像と技術スタック
2. リポジトリ直下の構成
3. frontend（Next.js）のファイル構成と役割
4. backend（Rails API）のファイル構成と役割
5. データモデルと API 仕様（現行コード準拠）
6. よくある変更と「どこを触るか」早見表
7. 起動手順・コマンド一覧
8. 保守上の注意点・既知の課題

凡例： 編集対象　日常的に手で編集するファイル　生成物　コマンドで自動生成・更新（手編集しない）　雛形　フレームワークの初期生成のまま／通常触らない

1. システム全体像と技術スタック

MateOrg は、講義スライド・課題・過去問などの**教材（Material）を登録・検索・編集・削除**し、ファイルを添付して管理する Web アプリです。フロントエンドとバックエンドを別アプリとして分離しています。



層	技術	役割
フロントエンド	Next.js 16.4.2（App Router, Turbopack） React 19.3 / TypeScript 5 Tailwind CSS 4（導入済み・現状ほぼ未使用）	画面表示、フォーム入力、検索条件・並び替えの管理、API 呼び出し

バックエンド	Ruby 4.0.7 / Rails 8.1.4 （`api_only`） Puma（Web サーバ）	REST API（JSON）、入力検証、DB 操作、ファイル添付の管理
データベース	PostgreSQL（gem `pg`）	教材レコード、Active Storage のメタデータを保存
ファイル保存	Active Storage（開発: ローカルディスク）	アップロードされた教材ファイルを `backend/storage/` に保存
品質・運用	ESLint / RuboCop / Brakeman / bundler-audit / Kamal / Docker	静的解析、セキュリティ検査、本番デプロイ（Rails 側は雛形のまま）

ポイント：ブラウザは Rails に直接アクセスせず、常に Next.js の `/api/...` を叩きます。Next.js がそれを `RAILS_API_URL`（既定 `http://localhost:3000`）へ転送するため、同一オリジン扱いとなり **CORS 設定は不要**になっています。

2. リポジトリ直下の構成

```
MateOrg/
├── .gitignore                … リポジトリ全体の Git 除外設定
├── README.md                 … プロジェクト名のみ（要拡充）
├── backend/                  … Rails API（→ 4章）
├── frontend/                 … Next.js アプリ（→ 3章）
├── docs/                     … 設計ドキュメント群
│   ├── requirrement.md       … 要件定義
│   ├── architecturre.md      … システム構成設計
│   ├── database-design.md    … DB 設計
│   ├── api-spec.md           … API 仕様
│   ├── screen-design.md      … 画面設計
│   └── setup-guide.md        … 環境構築手順
├── backend-tree.txt          … backend のツリー出力（スナップショット）
├── backend-source.txt        … backend のソース連結ダンプ（スナップショット）
├── frontend-next-tree.txt    … 旧 frontend-next のツリー出力（スナップショット）
└── frontend-next-source.txt  … 旧 frontend-next のソース連結ダンプ（スナップショット）
```

ファイル／フォルダ	役割・保守メモ
`docs/*.md`	要件・設計の一次資料。**コードを変えたら該当ドキュメントも更新**する。ファイル名に綴りの揺れ（requirrement / architecturre）があるので、リンク時は注意。docs は当初 React 単体＋科目テーブル等を想定した記述があり、現行コード（Next.js・materials テーブルのみ）と差異がある。
`*-tree.txt / *-source.txt`	過去に `tree` 等で出力したスナップショット。**Shift-JIS で保存**されておりエディタによって文字化けする。アプリの動作には無関係。最新ではないため、参照用でなければ削除してもよい。

3. frontend（Next.js）のファイル構成と役割

```
frontend/
├── src/
│   └── app/                  … App Router のルート（URL = フォルダ構成）
│       ├── layout.tsx        … 全ページ共通の HTML 骨格・フォント・メタ情報
│       ├── page.tsx          … 「/」のページ。教材管理 UI のすべて
│       ├── globals.css       … グローバル CSS（Tailwind 読み込み・色変数）
│       └── favicon.ico       … ブラウザタブのアイコン
├── public/                   … 静的ファイル（/file.svg のように直接配信）
│   └── file.svg, globe.svg, next.svg, vercel.svg, window.svg
├── next.config.ts            … Next.js 設定（API プロキシ・Turbopack 等）
├── package.json              … 依存パッケージと npm スクリプト
├── package-lock.json         … 依存のロックファイル
├── tsconfig.json             … TypeScript 設定（@/* エイリアス）
├── eslint.config.mjs         … ESLint 設定（eslint-config-next）
├── AGENTS.md                 … AI エージェント向け注意書き（next dev が自動追記）
├── README.md                 … create-next-app の標準 README
└── .gitignore                … node_modules / .next / .env* などを除外
```

3.1 ソースコード（src/app）

ファイル	役割・中身
src/app/page.tsx 編集対象	**アプリ本体（約 630 行）**。先頭に `'use client'` があり、クライアントコンポーネントとして動作。 **型定義：** `MaterialType`（slide / assignment / past_exam / notes / other）、`Material`（API レスポンス）、`MaterialForm`（フォーム入力）、`SortOrder`。 **定数：** `typeLabels`（種類コード → 日本語表示名）、`initialForm`。 **主な関数：** • `loadMaterials(filters)` … `GET /api/v1/materials`（検索条件をクエリに付与） • `handleSubmit` … 登録（POST）／編集（PATCH）。`FormData` で `material[title]` 等とファイルを送信 • `handleDelete` … 確認ダイアログ後 `DELETE` • `handleSearch` / `handleClearSearch` … 検索・条件クリア • `startEditing` / `cancelEditing` … 編集モード切替 • `openMaterialDetail` / `closeMaterialDetail` … 詳細表示 **画面構成（上から）：**見出し → 教材登録/編集フォーム → メッセージ/エラー → 教材検索 → 教材一覧（並び替え・カード表示）→ 教材詳細。 **並び替え**は API ではなくブラウザ側（`sortedMaterials`）で実施。 **スタイル**はファイル末尾のインラインスタイル定数（`pageStyle`, `sectionStyle`, `fieldStyle`, `inputStyle`, `cardStyle`）。
src/app/layout.tsx 編集対象	ルートレイアウト。`<html>` / `<body>` を出力し、Google Fonts（Geist / Geist Mono）を CSS 変数として読み込み、`globals.css` を適用。`metadata`（タブのタイトル・説明）を定義。 ※現状 `title: "Create Next App"`、`lang="en"` のまま。

`src/app/globals.css` 編集対象	`@import "tailwindcss"` で Tailwind v4 を読み込み、背景色・文字色の CSS 変数とダークモード時の色を定義。
`src/app/favicon.ico`	タブのアイコン。差し替えればそのまま反映。
`public/*.svg` 雛形	create-next-app のサンプル画像。現在どこからも参照されていないため削除可能。

3.2 設定ファイル

ファイル	役割・保守メモ
`next.config.ts` 編集対象	• rewrites：`/api/:path*` → `${RAILS_API_URL}/api/:path*`。Rails の URL を変えるときは環境変数 `RAILS_API_URL` を設定（未設定時 `http://localhost:3000`）。 • `cacheComponents: true`, `partialPrefetching: true` … Next.js 16 のキャッシュ／プリフェッチ機能。 • `turbopack.rules` … `*.css` を `@tailwindcss/turbopack` ローダーで処理。
`package.json`	スクリプト：`dev`（開発サーバ）、`build`（本番ビルド）、`start`（本番起動）、`lint`（ESLint）。依存：next 16.4.2 / react 19.3.0 / react-dom 19.3.0。開発依存：TypeScript, ESLint, Tailwind 等。
`package-lock.json` 生成物	`npm install` で更新。手編集しない。コミットは必要。
`tsconfig.json`	strict モード。`@/*` → `./src/*` のパスエイリアス（例：`import X from '@/components/X'`）。
`eslint.config.mjs`	Next.js 推奨の Lint ルール（Core Web Vitals / TypeScript）。
`AGENTS.md` 生成物	「この Next.js は学習データと異なる可能性があるので `node_modules/next/dist/docs/` を読むこと」という AI 向け注意書き。`next dev` が自動で再追加する。
`next-env.d.ts / .next/` 生成物	Next.js が自動生成する型定義・ビルド成果物。Git 管理外。

4. backend（Rails API）のファイル構成と役割

```
backend/
├── app/                          … アプリケーション本体
│   ├── controllers/
│   │   ├── application_controller.rb   … 全コントローラの親（ActionController::API）
│   │   ├── api/v1/materials_controller.rb … 教材 API（★本体）
│   │   └── concerns/             … 共通処理モジュール置き場（現在空）
│   ├── models/
│   │   ├── application_record.rb … 全モデルの親
│   │   ├── material.rb           … 教材モデル（★本体）
│   │   └── concerns/             … 共通処理モジュール置き場（現在空）
│   ├── jobs/application_job.rb   … バックグラウンドジョブの親（未使用）
│   ├── mailers/application_mailer.rb … メール送信の親（未使用）
│   └── views/layouts/mailer.*.erb    … メールのレイアウト（未使用）
├── config/                       … 設定（ルーティング・DB・環境ごとの設定）
├── db/                           … マイグレーションとスキーマ
├── bin/                          … 実行スクリプト（rails, setup, dev など）
├── test/                         … テスト（Minitest）
├── lib/tasks/                    … 独自 rake タスク置き場（現在空）
├── public/robots.txt             … クローラ向け設定
├── storage/                      … Active Storage の保存先（開発時のファイル実体）
├── log/, tmp/                    … ログ・一時ファイル（Git 管理外、.keep のみ）
├── vendor/                       … 外部ライブラリ置き場（空）
├── .github/                      … Dependabot・CI 設定（Rails 生成時のもの）
├── Gemfile / Gemfile.lock        … Ruby の依存 gem
├── Dockerfile / .dockerignore    … 本番用コンテナイメージ
├── config.ru, Rakefile           … Rack 起動設定・rake の入口
├── .ruby-version                 … Ruby バージョン（ruby-4.0.7）
└── .rubocop.yml                  … コードスタイル設定（rubocop-rails-omakase）
```

4.1 app/ — アプリケーションロジック

ファイル	役割・中身
app/controllers/api/v1/materials_controller.rb 編集対象	教材 API のすべて。`Api::V1` 名前空間（URL `/api/v1/...` に対応）。 • `index` … 作成日の降順で取得。`title`・`subject` は部分一致（`ILIKE`、大文字小文字無視）、`material_type` は完全一致で絞り込み。 • `create` … 新規作成。失敗時は `{ errors: [...] }` と 422。 • `update` … 更新（ファイルを送れば差し替え）。 • `destroy` … 削除し 204 を返す（添付ファイルも Active Storage が削除）。 • `material_json` … レスポンス整形。モデルの全カラムに `file_attached` / `file_url` / `file_name` を追加。 • `material_params`（private）… Strong Parameters。受け付ける項目は title, subject, material_type, description, file。

app/models/material.rb 編集対象	教材モデル（materials テーブル）。has_one_attached :file で 1 件のファイルを添付可能。 バリデーション：title・subject・material_type は必須、material_type は slide / assignment / past_exam / notes / other のいずれか。
app/controllers/application_controller.rb	全コントローラの基底クラス。認証など全 API 共通の処理を入れる場所（現在空）。
app/models/application_record.rb	全モデルの基底クラス（primary_abstract_class）。
app/jobs, app/mailers, app/views 雛形	Rails 生成時の雛形。現時点で未使用。

4.2 config/ — 設定

ファイル	役割・保守メモ
config/routes.rb 編集対象	URL とコントローラの対応表。namespace :api → :v1 → resources :materials, only: [index, create, update, destroy] と、ヘルスチェック GET /up。**show（1 件取得）は未定義。**
config/database.yml 編集対象	DB 接続設定。開発 DB は backend_development、ユーザー postgres、パスワードは環境変数 POSTGRES_PASSWORD。テストは backend_test。本番は primary / cache / queue / cable の 4 DB 構成（Solid 系 gem 用）。
config/storage.yml	Active Storage の保存先定義。local → storage/、test → tmp/storage/。S3 / GCS はコメントアウトのテンプレート。
config/application.rb	アプリ全体の設定。config.api_only = true（API 専用モード）、load_defaults 8.1。タイムゾーン等はここで設定。
config/environments/ development.rb / test.rb / production.rb	環境別設定。開発では active_storage.service = :local、メーラーの host を localhost:3000 に設定。
config/initializers/cors.rb	CORS 設定（**全てコメントアウト**）。Next.js 経由のプロキシ構成なので現状は不要。ブラウザから Rails を直接叩く構成にする場合は gem rack-cors と併せて有効化する。
config/initializers/ filter_parameter_logging.rb	パスワード等をログに出さないための設定。
config/initializers/inflections.rb	単数形/複数形の変換ルール（雛形）。
config/puma.rb	Web サーバ Puma の設定（スレッド数・ポート 3000 等）。
config/credentials.yml.enc	暗号化された秘密情報。復号には config/master.key（Git 管理外）が必要。編集は bin/rails credentials:edit。
config/cache.yml, queue.yml, cable.yml, recurring.yml 雛形	Solid Cache / Solid Queue / Solid Cable（DB をキャッシュ・ジョブキュー・WebSocket に使う仕組み）の設定。現状未使用。
config/deploy.yml 雛形	Kamal（Docker デプロイツール）の設定。サーバ IP 等はダミー値のまま。
config/ci.rb	bin/ci で実行するローカル CI の手順（セットアップ→RuboCop→脆弱性検査→テスト）。

`config/boot.rb`, `environment.rb`, `locales/en.yml`, `bundler-audit.yml`	起動処理・初期化・英語メッセージ・脆弱性検査の除外設定。通常触らない。

4.3 db/ — データベース

ファイル	役割・保守メモ
`db/migrate/20261018125448_create_active_storage_tables...rb`	Active Storage 用の 3 テーブル（blobs / attachments / variant_records）を作成。
`db/migrate/20261019064836_create_materials.rb`	`materials` テーブルを作成（title, subject, material_type, description, timestamps）。
`db/schema.rb` 生成物	現在の DB 構造。`bin/rails db:migrate` で自動更新。**直接編集せず、必ずマイグレーションを追加**する。コミットは必要。
`db/seeds.rb`	初期データ投入スクリプト（現在空）。`bin/rails db:seed` で実行。
`db/cache_schema.rb`, `queue_schema.rb`, `cable_schema.rb` 雛形	本番の Solid 系 DB 用スキーマ。

4.4 bin/・test/・その他

ファイル	役割
`bin/rails`, `bin/rake`	Rails コマンドの実行（`bin/rails s`, `bin/rails db:migrate` 等）。
`bin/setup`	初回セットアップ（bundle install・DB 準備・サーバ起動）。
`bin/dev`	開発サーバ起動。
`bin/rubocop`, `bin/brakeman`, `bin/bundler-audit`, `bin/ci`	コードスタイル検査、セキュリティ静的解析、gem 脆弱性検査、ローカル CI 一括実行。
`bin/jobs`, `bin/kamal`, `bin/thrust`, `bin/docker-entrypoint`	ジョブワーカー、デプロイ、本番用 HTTP プロキシ、コンテナ起動時の DB 準備。
`test/test_helper.rb`	テストの共通設定（並列実行・fixtures 読み込み）。
`test/models/material_test.rb`	Material モデルのテスト（**中身は空**）。
`test/fixtures/materials.yml`	テスト用データ（雛形の `MyString` のまま）。
`.github/workflows/ci.yml`, `.github/dependabot.yml`	GitHub Actions の CI と依存更新設定（Rails 生成時のもの）。
`Gemfile / Gemfile.lock`	依存 gem。rails, pg, puma, image_processing, solid_* 等。gem 追加後は `bundle install`。Gemfile.lock は生成物。
`Dockerfile`	本番用イメージ（Ruby 4.0.7-slim、libvips・postgresql-client 導入、非 root 実行、Thruster 経由で起動）。

5. データモデルと API 仕様（現行コード準拠）

5.1 materials テーブル

カラム	型	必須	説明
`id`	bigint	自動	主キー
`title`	string	○（モデル）	教材名
`subject`	string	○（モデル）	科目名（自由入力文字列）
`material_type`	string	○（モデル）	slide（講義スライド）／assignment（課題）／past_exam（過去問）／notes（ノート）／other（その他）
`description`	text	—	説明
`created_at` / `updated_at`	datetime	自動	登録日時／更新日時

※必須チェックはモデルのバリデーションのみで、DB 側に NOT NULL 制約はありません。添付ファイルは Active Storage の `active_storage_attachments`（name = "file"）経由で関連付けられます。

5.2 API エンドポイント

メソッド / パス	処理	リクエスト	レスポンス
`GET /api/v1/materials`	一覧・検索	クエリ（任意）：`title`, `subject`（部分一致）, `material_type`（完全一致）	200：教材の配列
`POST /api/v1/materials`	登録	multipart/form-data：`material[title]`, `material[subject]`, `material[material_type]`, `material[description]`, `material[file]`（任意）	201：教材／422：`{errors}`
`PATCH /api/v1/materials/:id`	更新	POST と同じ	200：教材／422：`{errors}`
`DELETE /api/v1/materials/:id`	削除	—	204
`GET /up`	ヘルスチェック	—	200 / 500

レスポンスの教材 JSON の例：

```
{
  "id": 1, "title": "情報数学 第1回", "subject": "情報数学", "material_type": "slide",
  "description": "…", "created_at": "2026-10-19T…", "updated_at": "2026-10-19T…",
  "file_attached": true,
  "file_url": "http://localhost:3000/rails/active_storage/blobs/redirect/…/slide.pdf",
  "file_name": "slide.pdf"
}
```


6. よくある変更と「どこを触るか」早見表

やりたいこと	触るファイルと手順
教材に項目を追加 （例：担当教員 teacher）	1. `cd backend` → `bin/rails g migration AddTeacherToMaterials teacher:string` → `bin/rails db:migrate`（schema.rb が自動更新） 2. 必須なら `app/models/material.rb` にバリデーション追加 3. `materials_controller.rb` の `material_params` の `permit` に `:teacher` を追加（**忘れると保存されない**） 4. 検索対象にするなら `index` に条件を追加 5. `frontend/src/app/page.tsx`：`Material`・`MaterialForm` 型、`initialForm`、`startEditing`、`handleSubmit` の `requestData.append`、フォーム・一覧・詳細の JSX を更新 6. `docs/database-design.md`, `docs/api-spec.md` を更新
教材の種類を追加	`material.rb` の `inclusion: { in: [...] }` と、`page.tsx` の `MaterialType` 型・`typeLabels` の**3 か所を必ずそろえる。**
新しい API を追加	`config/routes.rb` にルート追加 → コントローラにアクション追加（新リソースなら `app/controllers/api/v1/` に新ファイル）→ フロントから `/api/v1/...` で呼ぶ（プロキシは自動で効く）。
新しいページを追加	`frontend/src/app/<パス名>/page.tsx` を作成（例：`src/app/materials/[id]/page.tsx` → `/materials/1`）。共通部品は `src/components/` 等を作り `@/components/...` で import。
画面の見た目を変更	`page.tsx` 末尾のスタイル定数、または `globals.css`。Tailwind のクラスも使用可能。
タブのタイトル変更	`src/app/layout.tsx` の `metadata`。
Rails の URL/ポート変更	フロント：環境変数 `RAILS_API_URL`（`frontend/.env.local` 等）。Rails 側：`materials_controller.rb` の `material_json` に固定の host/port があるので併せて修正。
ファイル保存先を S3 に	`config/storage.yml` の amazon を有効化 → `config/environments/production.rb` の `active_storage.service` を変更 → `Gemfile` に `aws-sdk-s3` → credentials に鍵を登録。
DB 接続情報の変更	`config/database.yml`、または環境変数 `POSTGRES_PASSWORD` / `DATABASE_URL`。
テストを書く	`backend/test/models/material_test.rb`、API は `test/controllers/` や `test/integration/` に追加。`bin/rails test` で実行。

7. 起動手順・コマンド一覧

7.1 開発環境の起動

```
# 前提：PostgreSQL が起動しており、環境変数 POSTGRES_PASSWORD を設定済み

# --- backend（ポート 3000） ---
cd backend
bundle install
bin/rails db:prepare        # DB 作成 + マイグレーション
bin/rails s                 # http://localhost:3000

# --- frontend（別ターミナル） ---
cd frontend
npm install
npm run dev                 # 3000 が使用中なら自動で 3001 等になる
                            # ブラウザでは表示されたポートの URL を開く
```

ポート注意：Next.js の既定ポートも 3000 です。必ず **Rails を先に起動**するか、`npm run dev -- -p 3001` のようにポートを明示してください。Next.js が 3000 を取ると、`/api` の転送先が自分自身になり API が動きません。

7.2 よく使うコマンド

コマンド	用途
`bin/rails db:migrate`	マイグレーション適用（schema.rb 更新）
`bin/rails db:rollback`	直前のマイグレーションを取り消し
`bin/rails routes | grep material`	定義済みルートの確認
`bin/rails c`	Rails コンソール（例：`Material.count`）
`bin/rails test`	テスト実行
`bin/rubocop -a` / `bin/brakeman`	スタイル自動修正／セキュリティ検査
`npm run lint`（frontend）	ESLint
`npm run build` → `npm start`（frontend）	本番ビルドと起動（型エラーもここで検出）

8. 保守上の注意点・既知の課題

コード調査の時点で見つかった事項です。修正時の優先度の目安として **[高]**〜**[低]** を付けています。

[高] 教材登録（POST）が 500 エラーになる
`materials_controller.rb` の `create` で `status: :createdd` と綴りが誤っています（正しくは `:created`）。Rails が不明なステータスとして例外を出すため、**レコードは DB に保存されるがレスポンスはエラー**になり、画面には「教材を登録できませんでした」系のエラーが表示されます。

[中] ファイル URL のホストが固定

`material_json` 内の `rails_blob_url` が `host: "localhost", port: 3000` 固定です。本番や別ポートで動かすとファイルリンクが壊れます。`default_url_options` や環境変数で設定する形にすると安全です。また、このリンクは Next.js を経由せず Rails に直接アクセスします。

[中] テストがない／fixtures が不正値

`material_test.rb` は空で、`fixtures/materials.yml` の `material_type: MyString` はバリデーションに通らない値です。テストを書く前に fixtures を有効な値（例： `slide` ）へ直してください。

[中] CI が実行されない設定

`ci.yml` は `backend/.github/` にありますが、GitHub Actions はリポジトリ直下の `.github/` しか読みません。またトリガーが `master` ブランチ（このリポジトリは `main` ）です。CI を使うならルートへ移動し、`working-directory: backend` を指定してください。

[低] その他

- `page.tsx` が 1 ファイルに全機能を持つため肥大化しています。機能追加時は `MaterialForm` / `SearchForm` / `MaterialList` / `MaterialDetail` などのコンポーネントと、API 呼び出し用の `src/lib/api.ts` に分割すると保守しやすくなります。
- `layout.tsx` の `title` が「Create Next App」、`lang="en"` のまま。日本語アプリなので `lang="ja"` 推奨。
- `globals.css` のダークモード（背景が黒）と、`page.tsx` の文字色 `#222` 固定がぶつかり、ダークモードでは文字が読みにくくなります。
- Tailwind は導入済みですがスタイルはインライン定数で書かれており、方式が混在しています。どちらかに統一を推奨。
- `material_json` が `private` より上（public）にあります。ルートが無いので実害はありませんが、`private` 以下に移すのが慣例です。
- 件数表示「検索結果：N件」は検索していない時も表示されます。
- `docs/` の設計書は React 単体構成・科目テーブル等を前提にした記述があり、現行実装と一致していません。
- リポジトリ直下の `*-tree.txt` / `*-source.txt` は Shift-JIS の古いスナップショットです。
- `config/master.key` は Git 管理外です。紛失すると `credentials.yml.enc` を復号できないため、チーム内で安全に共有・保管してください。